

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ

Εργασία 2 - – Προγραμματισμός με OpenMP

Ονοματεπώνυμο : Κυριακάκος Νικόλαος Αριθμός Μητρώου : 1115202200083

Ονοματεπώνυμο : Γαυγιωτάκη Ευθυμία Αριθμός Μητρώου : 1115202200027

Περιεχόμενα

1 Δομή Κώδικα **2**

2 Εκτέλεση προγραμμάτων **3**
2.1 Μετρικές . . . 3
2.2 Αρχείο tests.sh . . . 3
2.3 Συμβάσεις κώδικα . . . 3

3 Άσκηση 2.1 **5**
3.1 Βήματα εκτέλεσης . . . 5
3.2 Αποτελέσματα εκτέλεσης . . . 5

4 Άσκηση 2.2 **7**
4.1 Παραλληλοποίηση βρόγχων (Ερώτημα εκφώνησης) . . . 7
4.2 Βήματα εκτέλεσης . . . 7
4.3 Αποτελέσματα εκτέλεσης . . . 7

1 Δομή Κώδικα

Ο κώδικας αποτελείται από τους εξής φακέλους:

- Bin: εδώ τοποθετούνται τα εκτελέσιμα αρχεία που προκύπτουν από τη μεταγλώττιση των προγραμμάτων
- Build: εδώ τοποθετούνται τα αντικείμενα αρχεία που προκύπτουν από τη μεταγλώττιση των προγραμμάτων
- Results: εδώ τοποθετούνται τα αρχεία που περιέχουν τα αποτελέσματα του προγραμμάτων, τα γραφήματα καθώς και ένα αρχείο που περιέχει τις πληροφορίες του συστήματος που ζητούνται από την εκφώνηση.
- Include: στον φάκελο αυτό υπάρχουν τα αρχεία κεφαλίδας
- Src: στον φάκελο αυτό βρίσκεται ο πηγαίος κώδικας των προγραμμάτων

Περιλαμβάνονται επίσης τα εξής αρχεία:

- Makefile
- graphs.py: το πρόγραμμα αυτό χρησιμοποιείται για να διαβάσει και να αποθηκεύσει κατάλληλα τα αποτελέσματα που βρίσκονται στα αρχεία που δημιουργήθηκαν από την εκτέλεση των προγραμμάτων και να δημιουργήσει τα αντίστοιχα γραφήματα.
- tests.sh: το αρχείο αυτό είναι εκείνο το οποίο χρησιμοποιήθηκε για τη δημιουργία και εκτέλεση των προγραμμάτων.

2 Εκτέλεση προγραμμάτων

2.1 Μετρικές

Τα προγράμματα δοκιμάστηκαν στο μηχάνημα linux19 της σχολής.
Το μηχάνημα αυτό έχει 4 πυρήνες. Συνεπώς όπως αναφέρθηκε και στο μάθημα, τα τεστ που διενεργήθηκαν έγιναν με αριθμό νημάτων μέχρι 4 * πυρήνες, δηλαδή 16.
Συνεπώς, σε όλα τα τεστ (και στα αποτελέσματα που παρατίθενται στη συνέχεια) έχουν χρησιμοποιηθεί οι αριθμοί νημάτων 1 2 4 8 16.
Κάθε πρόγραμμα εκτελέστηκε με διαφορετικό ορισμάτων 4 φορές το καθένα. Αυτό είχε ως σκοπό να μετρηθεί ο μέσος όρος των αποτελεσμάτων ώστε να αποφευχθούν διάφορες αστοχίες. Τα αποτελέσματα που ακολουθούν παρακάτω είναι όλα αυτά των μέσων όρων. Εκτελώντας ωστόσο το αρχείο tests.sh προκύπτουν επίσης όλα τα αποτελέσματα αναλυτικά στο φάκελο results.
Τα χαρακτηριστικά του υπολογιστικού συστήματος που χρησιμοποιήθηκε (linux19) είναι τα εξής:

Χαρακτηριστικό	**Λεπτομέρειες**
Hostname	linux19
CPU Model	Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
CPU Cores	4
OS Version	Ubuntu 20.04.6 LTS
GCC Version	gcc (Ubuntu 9.4.0-1ubuntu1 20.04.2) 9.4.0

2.2 Αρχείο tests.sh

Για την εκτέλεση των προγραμμάτων και στη συνέχεια τη δημιουργία των γραφημάτων χρησιμοποιήθηκε το αρχείο tests.sh.
Ουσιαστικά μέσω των συναρτήσεων που περιέχει κάνει τα εξής:

1. καθαρίζει τυχόν προηγούμενα αποτελέσματα εκτελέσεων και μεταγλωττίσεων
2. μέσω εντολών bash μας δίνει τις πληροφορίες του συστήματος που εκτελείται το πρόγραμμα (βρίσκονται στο αρχείο system_info.txt στον φάκελο results)
3. μεταγλωττίζει τον πηγαίο κώδικα και δημιουργεί τα αντίστοιχα εκτελέσιμα προγράμματα
4. εκτελεί τον κώδικα του αρχείου graphs.py ο οποίος παράγει τα γραφήματα που επιθυμούμε

2.3 Συμβάσεις κώδικα

1. Υπάρχει ένα αρχείο common.c στον φάκελο src, στο οποίο υπάρχουν ορισμένες συναρτήσεις διαχείρισης λάθους.
   Αυτές εμφανίζουν το λάθος στο χρήστη (αναλόγως αν είναι λάθος συστήματος ή λάθος για παράδειγμα ορισμάτων) και καλώντας της συνάρτηση exit τερματίζουν. Επίσης, περιέχονται κι ορισμένες ακόμα χρήσιμες συναρτήσεις όπως η open_file η οποία ανοίγει το αρχείο με το δοσμένο όνομα.
2. Ο κώδικας για την πρώτη άσκηση βρίσκεται στον φάκελο με όνομα game_of_life και για τη δεύτερη άσκηση αντίστοιχα στον φάκελο gauss.
3. Χρησιμοποιείται το αρχείο tests.sh για εκτέλεση όλων των προγραμμάτων ή εφόσον έχουν μεταγλωττιστεί τα προγράμματα, καλούνται με εντολή όπως η ./bin/game_of_life
4. Τα αρχεία αποτελεσμάτων έχουν αντίστοιχο όνομα με αυτό των προγραμμάτων με κατάληξη .txt
   Ομοίως, τα γραφήματα έχουν αντίστοιχο όνομα με αυτό των προγραμμάτων με κατάληξη .png.

5. Όλα τα παράλληλα προγράμματα εκτελούνται μέχρι και με 4 νήματα στο παραδοτέο καθώς το σύστημα που χρησιμοποιήθηκε υποστηρίζει μέχρι 4 νήματα για OpenMP, πληροφορία που πήραμε από την συνάρτηση omp_get_max_threads() της βιβλιοθήκης omp. Παρ' όλα αυτά παρακάτω παραθέτουμε τα αποτελέσματα εκτέλεσης με τη χρήση μέχρι και 16 νημάτων, όπου βλέπουμε την απόδοση να μειώνεται. Αυτό συμβαίνει διότι η φυσική υποστήριξη των νημάτων από το υλικό περιορίζεται, και έτσι η περαιτέρω αύξηση των νημάτων δημιουργεί πρόσθετη συμφόρηση.

3 Άσκηση 2.1

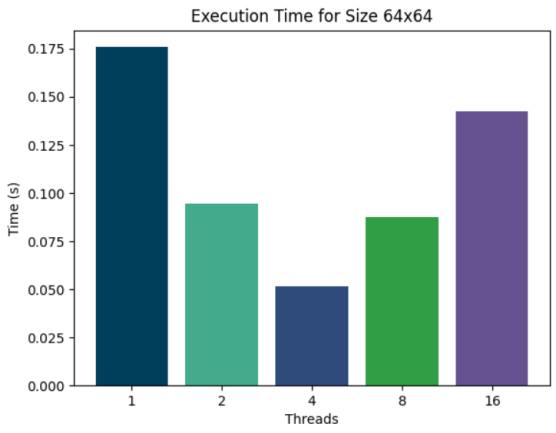
3.1 Βήματα εκτέλεσης

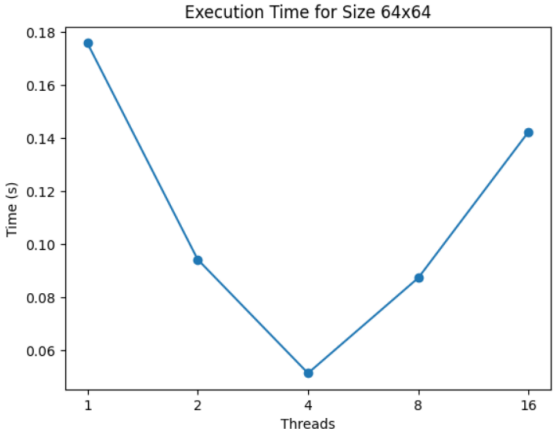
1. Ελέγχεται ότι τα ορίσματα είναι σωστά.
2. Δημιουργείται ένα αρχείο το οποίο η συνάρτηση γεμίζει τυχαία με τα δεδομένα του αρχικού grid. Με βάση τις συντεταγμένες καθορίζεται δηλαδή ποια κελιά θα είναι true (alive) και false αντίστοιχα.
3. Με την συνάρτηση create_grid δεσμεύεται η απαραίτητη μνήμη για το πρώτο grid και αρχικοποιούνται όλες οι θέσεις με την τιμή DEAD και με τη συνάρτηση fill_grid του δίνουμε τις τιμές που θέλουμε.
4. Δημιουργείται ένα επιπλέον grid έτσι ώστε να χρησιμοποιούμε τους δύο πίνακες εναλλάξ σε κάθε update, προκειμένου να μην δεσμεύουμε και ελευθερώνουμε μνήμη σε κάθε γύρο του παιχνιδιού.
5. Η εκτέλεση του παιχνιδιού ξεκινάει από την κλήση της συνάρτησης execute_game_of_life η οποία υπολογίζει και επιστρέφει τον χρόνο εκτέλεσης, για τον δοσμένο αριθμό γενιών καλεί την σωστή συνάρτηση για το update του grid ανάλογα με το αν απαιτείται παράλληλη ή σειριακή εκτέλεση και ενημερώνει τις μεταβλητές των δύο πινάκων για τον επόμενο γύρο.
6. Η συνάρτηση update_grid_parallel ενημερώνει ένα πλέγμα για το επόμενο βήμα. Χρησιμοποιεί την οδηγία # pragma omp για να διαμοιράσει τους υπολογισμούς σε πολλαπλά threads, ώστε να υπολογιστεί το νέο πλέγμα next_grid από το τρέχον πλέγμα. Το schedule(static) αναθέτει ίσα τμήματα της βρόχου στα threads. Η επιλογή collapse(2) συγχωνεύει τις δύο εμφωλευμένες βρόχους σε μία, ώστε να γίνεται παράλληλη εκτέλεση και στις δύο διαστάσεις του πλέγματος. Για κάθε κελί, μετρά τους ζωντανούς γείτονες και αποφασίζει αν θα παραμείνει ζωντανό ή θα πεθάνει σύμφωνα με τους κανόνες.
7. Εκτυπώνονται τα αποτελέσματα της εκτέλεσης.
8. Ελευθερώνεται όλος ο δεσμευμένος χώρος μνήμης.

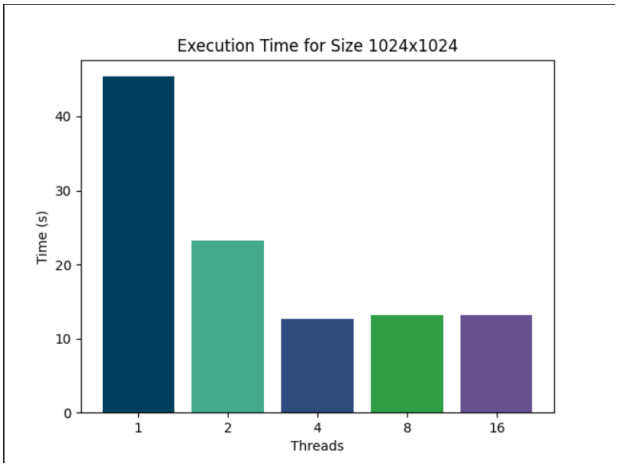
3.2 Αποτελέσματα εκτέλεσης

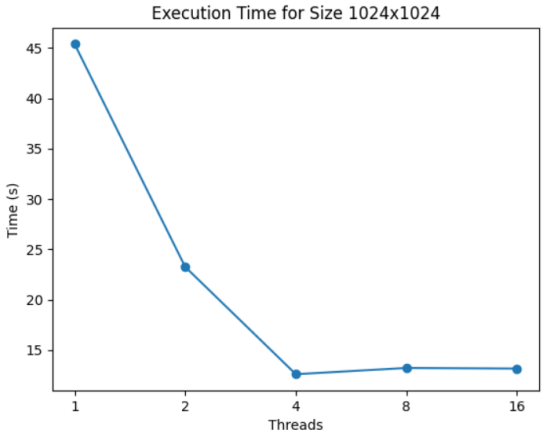
Τα αποτελέσματα δείχνουν ότι ο χρόνος εκτέλεσης αυξάνεται με το μέγεθος του πλέγματος, καθώς περισσότερα κελιά πρέπει να υπολογιστούν. Η παραλληλοποίηση μειώνει σημαντικά τον χρόνο, ειδικά για μεγάλα πλέγματα, αξιοποιώντας πολλούς πυρήνες. Περισσότερα νήματα βελτιώνουν την απόδοση, αλλά το όφελος μειώνεται για μικρά πλέγματα λόγω του overhead. Αντίθετα, στη σειριακή εκτέλεση, ο χρόνος είναι αναμενόμενα πολύ μεγαλύτερος.

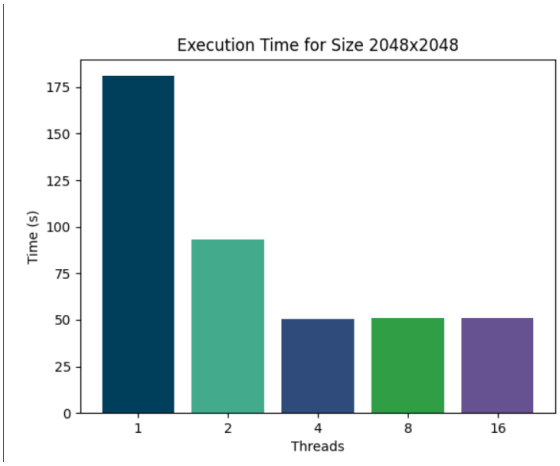
Παρακάτω φαίνονται τα γραφήματα που παρουσιάζουν τα αποτελέσματα εκτέλεσης του προγράμματος για διαφορετικούς αριθμούς γενιών.

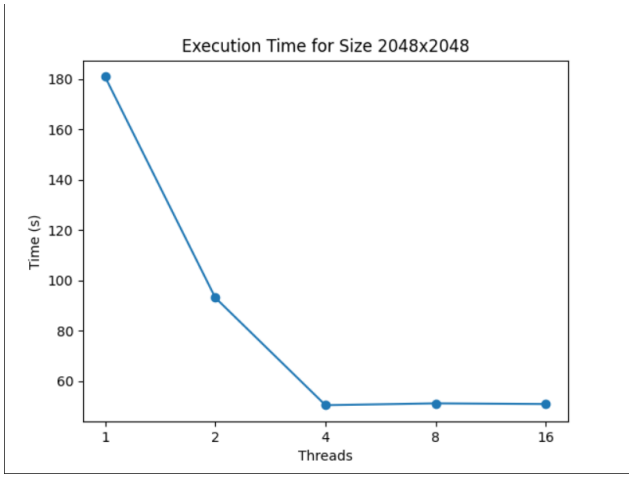





Execution Time for Size 1024x1024
Time (s)
0
10
20
30
40
1
2
4
8
16
Threads


Execution Time for Size 1024x1024
Time (s)
15
20
25
30
35
40
45
1
2
4
8
16
Threads


Execution Time for Size 2048x2048
Time (s)
0
25
50
75
100
125
150
175
1
2
4
8
16
Threads


Execution Time for Size 2048x2048
Time (s)
60
80
100
120
140
160
180
1
2
4
8
16
Threads

4 Άσκηση 2.2

4.1 Παραλληλοποίηση βρόγχων (Ερώτημα εκφώνησης)

1. Κατά γραμμή αλγόριθμος: Δεν μπορούμε να παραλληλοποιήσουμε τον εξωτερικό βρόγχο, διότι κάθε επανάληψη εξαρτάται από τις προηγούμενες. Η τιμή του x[row] εξαρτάται από τις τιμές του x[col] για όλες τις στήλες col > row. Αυτό σημαίνει ότι δεν μπορούμε να υπολογίσουμε την τιμή του x[row] πριν υπολογιστούν όλες οι προηγούμενες τιμές του x[col] με μεγαλύτερο col.

   Δεν υπάρχουν εξαρτήσεις δεδομένων μέσα στον εσωτερικό βρόχο, επομένως μπορεί να παραλληλοποιηθεί.

2. Κατά στήλη αλγόριθμος: Αντίστοιχα με προηγουμένως η τιμή του x[col] εξαρτάται από τις τιμές του x[row] για όλες τις γραμμές row < col. Αυτό σημαίνει ότι δεν μπορούμε να υπολογίσουμε την τιμή του x[col] πριν υπολογιστούν όλες οι προηγούμενες τιμές του x[row] με μικρότερο col. Επομένως δεν παραλληλοποιούμε τον εξωτερικό βρόγχο.

   Στον εσωτερικό βρόγχο δεν βλέπουμε εξαρτήσεις των τιμών μεταξύ των επαναλήψεων οπότε μπορεί να παραλληλοποιηθεί.

4.2 Βήματα εκτέλεσης

1. Ελέγχεται ότι τα ορίσματα είναι σωστά.
2. Καλείται η συνάρτηση init_array η οποία δεσμεύει μνήμη για τον πίνακα A και τα διανύσματα x και b και τα αρχικοποιεί με τυχαίες τιμές με το μέγεθος που δίνεται στα αρχικά ορίσματα της main.
3. Καλείται η συνάρτηση execute_substitution στην οποία αποφασίζεται αν θα εκτελεστεί η μέθοδος κατά στήλη ή κατά γραμμή και με παράλληλη ή σειριακή υλοποίηση και επιστρέφεται ο χρόνος που χρειάστηκε για την εκτέλεση.
4. Από την execute_substitution καλείται η σωστή συνάρτηση που εκτελεί τη αντικατάσταση.
5. Γίνεται η εμφάνιση των αποτελεσμάτων μέσω της συνάρτησης print_results
6. Καλείται η συνάρτηση free_memories η οποία απελευθερώνει όλες τις θέσεις μνήμης που έχουμε δεσμεύσει.

4.3 Αποτελέσματα εκτέλεσης

Τα αποτελέσματα δείχνουν ότι το static schedule αποδίδει καλύτερα όταν οι εργασίες είναι ισοκατανεμημένες, ενώ το dynamic και το guided προτιμώνται σε ανομοιόμορφα προβλήματα ή με μεγάλο αριθμό threads. Το guided είναι συχνά πιο αποδοτικό από το dynamic λόγω μειωμένου overhead. Με 1 thread, όλοι οι χρονοπρογραμματισμοί έχουν παρόμοιο χρόνο, ενώ με περισσότερα threads το static παρουσιάζει καλύτερη απόδοση. Η μέθοδος by_column τείνει να επωφελείται περισσότερο από τον παραλληλισμό σε σχέση με την by_line, ιδιαίτερα για μεγαλύτερους αριθμούς threads, καθώς κατανέμει τις εργασίες πιο αποτελεσματικά.

Παρακάτω φαίνονται τα γραφήματα που παρουσιάζουν τα αποτελέσματα εκτέλεσης του προγράμματος για διαφορετικά μεγέθη, με σύγκριση του αλγορίθμου κατά σειρά με τον αλγόριθμο κατά στήλη και με σύγκριση των ορισμάτων schedule():

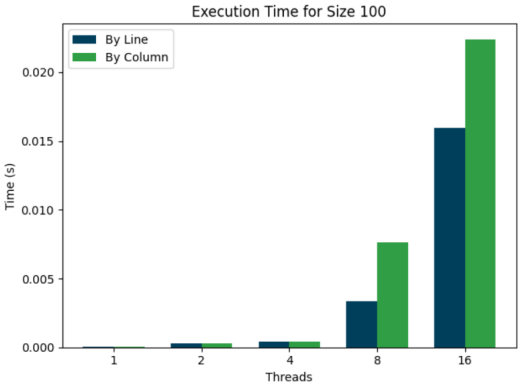


Figure 1: Σύγκριση αλγορίθμων για μέγεθος 100

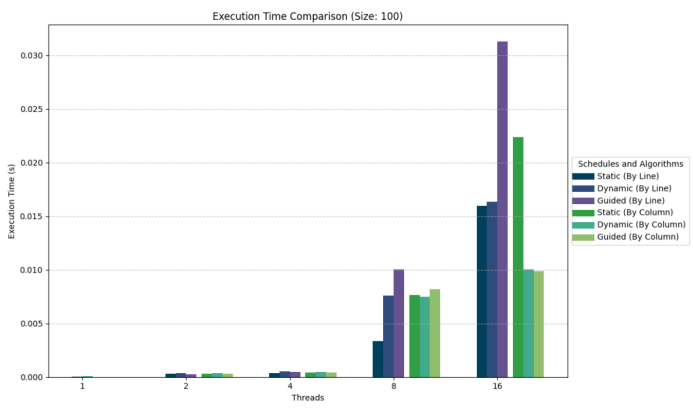


Figure 2: Σύγκριση χρονοδιαγραμμάτων για μέγεθος 100

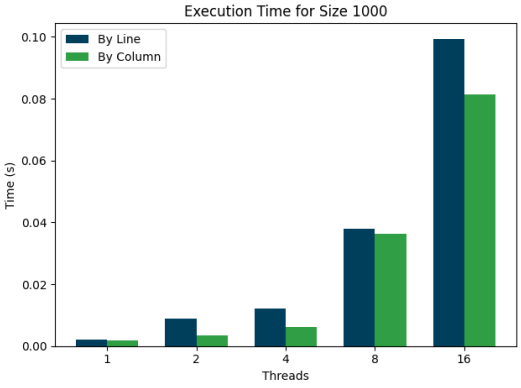


Figure 3: Σύγκριση αλγορίθμων για μέγεθος 1000

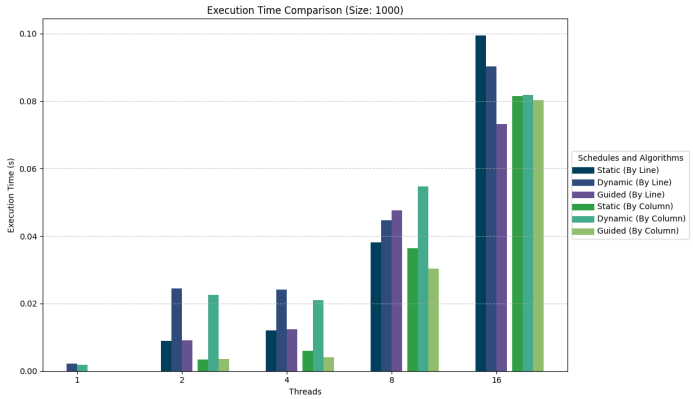


Figure 4: Σύγκριση χρονοδιαγραμμάτων για μέγεθος 1000

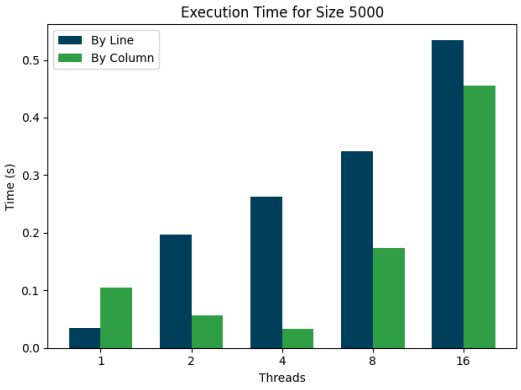


Figure 5: Σύγκριση αλγορίθμων για μέγεθος 5000

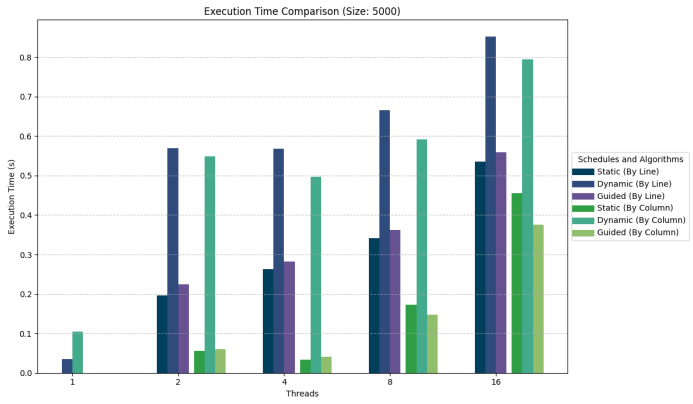


Figure 6: Σύγκριση χρονοδιαγραμμάτων για μέγεθος 5000

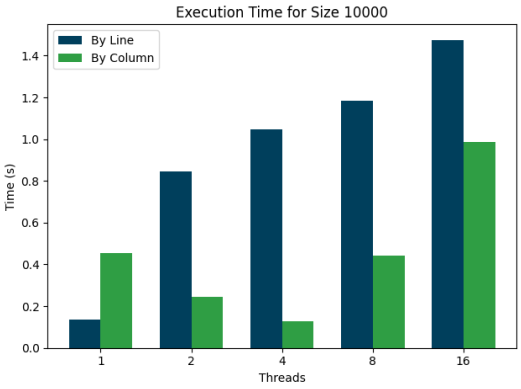


Figure 7: Σύγκριση αλγορίθμων για μέγεθος 10000

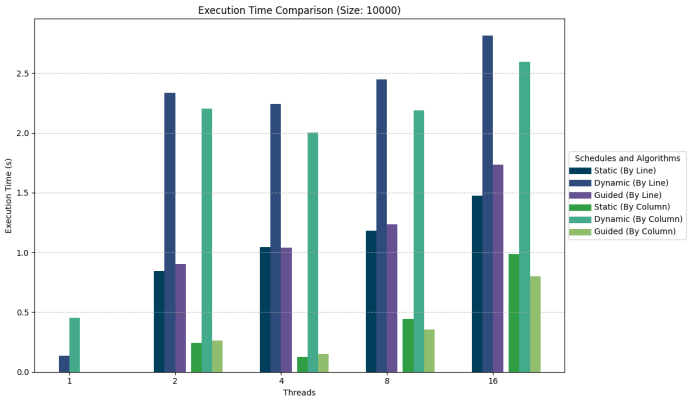


Figure 8: Σύγκριση χρονοδιαγραμμάτων για μέγεθος 10000